

El Modelo Orientado a Documentos

Autor

Martín Vladimir Alonso Sierra Galvis

Gestión de Datos, Maestría en Ciencia de Datos
Pontificia Universidad Javeriana Cali

Versión 2.0
Santiago de Cali, febrero de 2023

Tabla de contenido

Entendiendo el Modelo	2
Colecciones	2
Documentos	2
¿Cómo se relacionan los datos?	5
Anidaciones	5
Enlaces	6
ObjectId	8
Documentación del Modelo	9
Referencias	11

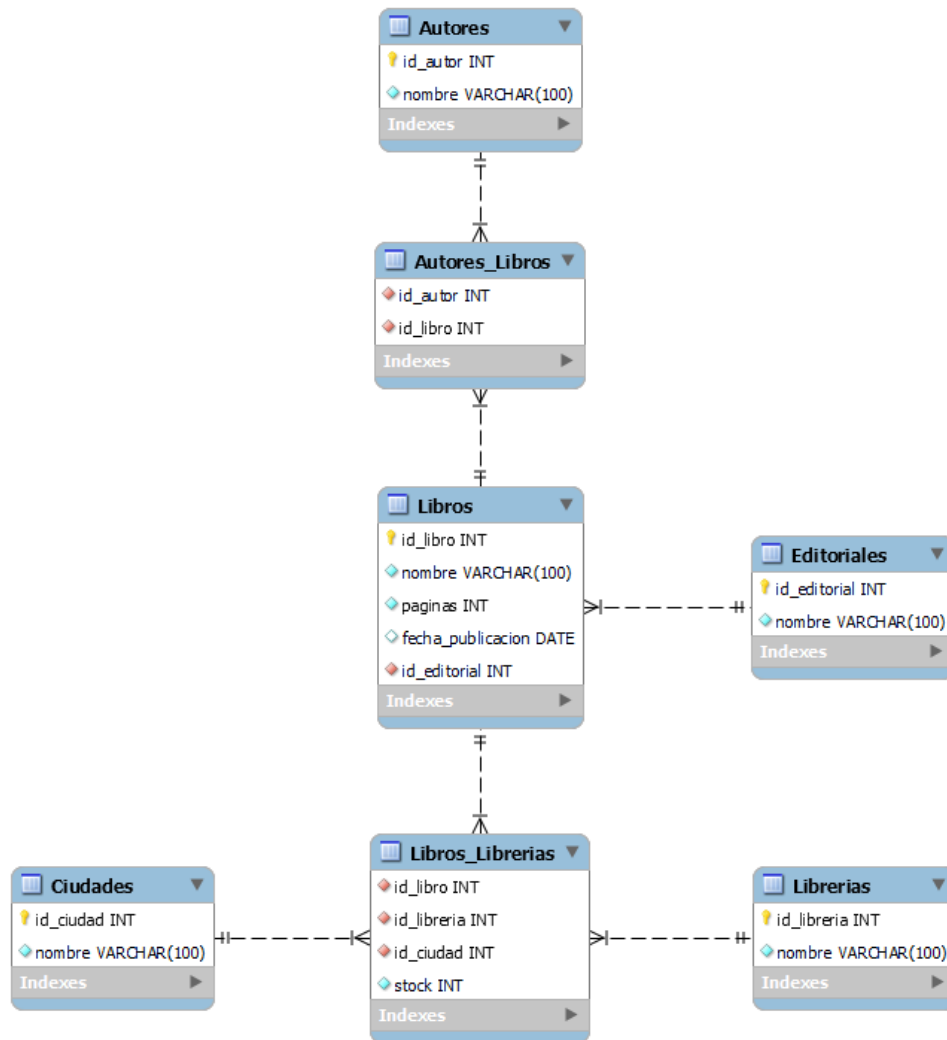
Entendiendo el Modelo

Colecciones

Como ya lo hemos mencionado, MongoDB es un Gestor de Bases de Datos Orientada a Documentos. Esto quiere decir que, a diferencia de lo que ocurría con los Gestores de Bases de Datos Relacionales, los datos no son almacenados en estructura de tablas. Por el contrario, MongoDB es de naturaleza **semiestructurada**, es decir, los datos no siguen una organización a priori estricta, sino que se organizan a partir de ciertas características que puedan compartir. La flexibilidad de MongoDB con respecto a la organización de los datos es tal que algunos incluso afirman que este gestor es de carácter no estructurado. Ahora bien, en MongoDB, los datos se organizan en lo que se denomina documentos y estos, a su vez, se almacenan en colecciones. Básicamente, las colecciones son agrupaciones de documentos y las podemos equiparar con las tablas de las bases de datos relacionales. Y así como una base de datos como Oracle puede albergar una o varias tablas, una base de datos MongoDB puede albergar una o varias colecciones.

Documentos

Los documentos son el contenedor de datos principal en las Bases de Datos Orientadas a Documentos, en este caso, MongoDB. Así como las colecciones son equivalentes a las tablas, los documentos son equivalentes a los registros o filas en la tabla. Los datos en un documento se almacenan con el formato BSON o representación binaria del formato JSON. Esto le confiere a MongoDB la facultad para utilizar una mayor variedad de tipos de datos de los que normalmente se utilizan en un archivo JSON puro. Al igual que sucede con los archivos JSON que vimos en la primera unidad de este módulo, los datos en un archivo BSON de MongoDB se organizan mediante parejas **clave-valor**. Para entender esto de una mejor manera, tomemos como ejemplo la base de datos que trabajamos en la anterior unidad, la base de datos de la aplicación de venta de libros. A continuación podemos ver el modelo relacional de dicha base de datos.



Del anterior modelo centrémonos en la tabla Libros. Como vimos en la sección anterior, la misma se convertiría en una colección en MongoDB. Ahora bien, sabemos que los datos de los libros, organizados en la tabla Libros, se verían así, en forma de matriz.

id_libro	nombre	páginas	fecha_publicacion	id_editorial
1	Fundación	230	01/01/51	2
2	Fundación e Imperio	230	01/01/52	2
3	Segunda Fundación	230	01/01/53	5

La anterior organización cambia drásticamente en los documentos de MongoDB. Veamos cómo se organizaría el primer registro de la tabla, llamado Fundación, usando el formato BSON.

```
{
  "id_libro": 1,
  "nombre": "Fundación",
  "paginas": 230,
  "fecha_publicacion": "01/01/51",
  "id_editorial": 2
},
```

Como podemos observar, aquí no tenemos filas ni columnas; aquí lo que debemos tener muy claro es el concepto clave-valor, de forma que sepamos cómo acceder los datos correctamente. En este ejemplo, **id_libro**, **nombre**, **paginas**, **fecha_publicacion** y **id_editorial** se denominan claves o campos; mientras que cada uno de los datos asociados a ellos se conocen como valores. Como conocimiento general, algo que resulta interesante es que, a diferencia de lo que ocurre con las bases de datos relacionales, en el proceso de diseño e implementación de una base de datos orientada a documentos, los Ingenieros de Datos encargados de esta etapa no tienen la necesidad de especificar el tipo exacto de dato que tendrá cada clave, sino que, al asociar un valor a esta, el mismo gestor interpreta el tipo de dato de manera automática.

Al ser de naturaleza semiestructurada, los documentos de MongoDB tienen una característica que, aunque les otorga un nivel de flexibilidad muy alto, puede ocasionar muchos problemas al momento de acceder y procesar datos: no es obligación que todos los documentos en una colección tengan exactamente la misma organización. Veamos la siguiente imagen.

```
{
  "id_libro": 2,
  "nombre": "Fundación e Imperio",
  "paginas": 230
}
```

El documento corresponde al segundo registro de la tabla Libros, Fundación e Imperio. Lo más notable es que este nuevo documento no tiene el mismo número de parejas clave-valor que el primer documento. Contrario a lo que podríamos pensar, MongoDB no pone problema al organizar los datos de esta forma; en ese sentido el gestor no es estricto. Es por esto que, si queremos consultar datos de una base de datos MongoDB para que sirvan de insumo a un algoritmo, por ejemplo, de Machine Learning, debemos ser muy cuidadosos; posiblemente debamos aplicar procesos de limpieza y preparación de datos para reparar posibles inconsistencias en los mismos. Debido a este tipo de situaciones, la limpieza y preparación de datos es un tema muy importante en la Gestión de Datos en el contexto de la Ciencia de Datos, y en el que profundizaremos en el próximo módulo del curso.

¿Cómo se relacionan los datos?

En un principio podríamos sorprendernos por el título de esta sección. Al ser las bases de datos orientadas a documentos, como MongoDB, una base de datos de tipo NoSQL, lo primero que pensamos es que aquí no existen las relaciones entre datos. Nada más alejado de la realidad. MongoDB cuenta con soporte para implementar relaciones entre los datos de los diferentes documentos, lo que permite una mejor organización de los mismos en la estructura. Obviamente, los Ingenieros de Datos y demás profesionales encargados del diseño y creación de una base de datos MongoDB deben tener cuidado al relacionar los datos pues, en este tipo de bases de datos, se debe priorizar el rendimiento en las consultas, factor que puede verse afectado por las relaciones, especialmente las relaciones cruzadas. En nuestro caso, como científicos de datos, debemos saber que, cuando consultamos datos en una base de datos MongoDB, podemos encontrarnos dos tipos de relaciones diferentes: anidaciones y enlaces.

Anidaciones

Las anidaciones son, como su nombre lo indica, la inserción de todos los datos de un documento dentro de otro. Esto se logra mediante el tipo de datos **Object** de BSON. Si observamos nuevamente el documento con los datos del primer libro, Fundación, podremos notar que la clave `id_editorial` tiene como valor el entero 2. Esto se debe a que, en el modelo relacional, dicho valor representa una llave foránea, es decir, una conexión con la llave primaria de la tabla Editoriales. Esto, a su vez, conforma lo que se conoce como una

relación 1 a muchos (1:M). La cuestión es que en MongoDB, este número no nos dice mucho, por lo que una posible solución es anidar los datos de la editorial y renombrar la clave, tal como se muestra en la siguiente imagen.

```
{
  "id_libro": 1,
  "nombre": "Fundación",
  "paginas": 230,
  "fecha_publicacion": "01/01/51",
  "editorial": {
    "id_editorial": 2,
    "nombre": "Gnome Press"
  }
},
```

La gran ventaja de las anidaciones es que podremos obtener gran cantidad de datos relacionados con una consulta muy simple. Es común encontrarlas cuando la base de datos está pensada para tener un mejor desempeño en lectura a costa de tener datos duplicados (la redundancia de datos no es algo que se desee evitar en bases de datos NoSQL); los datos anidados no crecen desmesuradamente ni cambian de forma constante; y se desea representar relaciones uno a uno y uno a muchos.

Enlaces

Los enlaces son el segundo tipo de relaciones con las que nos podemos topar cuando realicemos consultas a una base de datos MongoDB. Su enfoque es muy parecido al uso de llaves primarias y llaves foráneas en los modelos relacionales. La idea es utilizar el identificador de un documento en otro. Pongamos un ejemplo para explicarlo un poco mejor. En el modelo relacional de la base de datos de la aplicación de venta de libros teníamos una relación muchos a muchos (N:M) entre la tabla Libros y la tabla Autores. Esta relación se implementa mediante la creación de una tabla intermedia Autores_Libros, misma que contenía las dos llaves foráneas que se conectaban a las llaves primarias de las tablas antes mencionadas. Ahora bien, en una base de datos orientada a documentos bien diseñada, no encontraremos, para este caso, un documento intermedio con dos identificadores, sino un arreglo o lista de identificadores en alguno de los documentos implicados en la relación.

Supongamos que la siguiente imagen muestra el documento que contiene los datos de Isaac Asimov, autor del libro Fundación.

```
{
  "id_autor": 1,
  "nombre": "Isaac Asimov"
}
```

Como la relación que existe entre Isaac Asimov y Fundación tiene una cardinalidad N:M, podríamos toparnos con la representación que se muestra a continuación.

```
{
  "id_libro": 1,
  "nombre": "Fundación",
  "paginas": 230,
  "fecha_publicacion": "01/01/51",
  "editorial": {
    "id_editorial": 2,
    "nombre": "Gnome Press"
  },
  "autores": [1]
},
```

Donde la clave **autores** tiene como valor una lista de identificadores que referencian documentos de autores. Así mismo, es válido tener una clave **libros** en el documento del autor, cuyo valor sería igualmente una lista de identificadores, pero esta vez referenciando documentos de libros. La ventaja de los enlaces radica en que se evita la redundancia de datos, pero trae consigo un pequeño inconveniente: tendremos que utilizar una consulta cruzada de datos, lo que implica una penalización en el rendimiento al realizar el acceso. Nos enfrentaremos a enlaces cuando se desee evitar un número muy grande de datos duplicados en la base de datos; los datos crecen desmesuradamente o cambian muy a menudo; y se desea representar relaciones muchos a muchos.

ObjectId

Para finalizar, es importante que tengamos en cuenta una funcionalidad muy interesante del gestor MongoDB: cuando se crean documentos nuevos en alguna colección de la base de datos, el gestor se encarga de colocar automáticamente un identificador único, en la clave **_id**, que hace las veces de llave primaria, por lo que es muy posible que al consultar un documento nos encontremos una estructura así.

```
{
  "_id": "65890dg764k01283bd0471b0",
  "nombre": "Fundación",
  "paginas": 230,
  "fecha_publicacion": "01/01/51",
  "editorial": {
    "_id": "34f5h73056f2gm912350hd04",
    "nombre": "Gnome Press"
  },
  "autores": [
    "762897f821078bd15167189f2bd05"
  ]
},
```

Estos identificadores se conocen como ObjectId y con ellos ya no es necesario para el profesional encargado del diseño y creación de la base de datos asignar manualmente una llave única. Son estos ObjectId los que utiliza el gestor para representar las relaciones que vimos anteriormente, por lo que es un elemento muy importante en MongoDB.

Documentación del Modelo

En la anterior unidad vimos que para nosotros, como científicos de datos, es sumamente útil contar con un Modelo Relacional que nos facilite el proceso de consulta de datos. Esto mismo se cumple cuando trabajamos con bases de datos de documentos como MongoDB. El problema es que no existe un diagrama que represente el Modelo Orientado a Documentos. Podemos tratar de solventar esta carencia con una estrategia diferente: usar la documentación del modelo. Ingenieros de Software y de Datos cuidadosos siempre documentan sus proyectos y, en toda esa documentación, muy seguramente hay un capítulo o sección para la base de datos que nosotros podemos aprovechar. Ahora bien, el hecho de ser científicos de datos no nos impide crear una documentación propia del modelo en caso que no exista una. Recordemos que los científicos de datos deben estar en capacidad de comprender la organización de los datos en una base de datos. La documentación la podemos hacer si tenemos a mano un Modelo Relacional (esto en el caso en que la base de datos MongoDB se haya creado a partir de una base de datos relacional), o simplemente realizando un proceso de exploración de la base de datos, siendo esta última la opción que, muy seguramente, usaremos más a menudo. En caso de optar por la primera opción, debemos tener muy en cuenta todo lo expuesto anteriormente en esta lectura. A continuación se muestra una posible documentación de la colección Libros y la estructura de sus documentos.

```
Collection: Libros
[Document{
  _id: ObjectId,
  nombre: String,
  paginas: Integer,
  fecha_publicacion: Date,
  editorial: [{
    _id: ObjectId,
    nombre: String
  }]
}]
```

Debemos tener muy presente que la anterior es simplemente una **propuesta** de documentación, pues no existe un consenso respecto a la forma correcta de documentar una base de datos MongoDB. En esta propuesta colocamos el nombre de la colección y, seguidamente, usando corchetes, especificamos que dicha colección estará compuesta por una lista de documentos. Finalmente, dentro de las llaves que se encuentran después de la palabra Document colocamos las claves o campos que debería tener cada documento junto a su respectivo tipo de dato. Este estilo de documentación puede ser de gran ayuda para cualquier profesional que trabaje con el Modelo de Documentos pues ayuda a comprender fácilmente la organización de los datos en la base de datos.

Nota final:

A modo de ejercicio personal y antes de ver los videos de la unidad: imaginemos que la aplicación de venta de libros ahora está soportada por una base de datos MongoDB. Usando como base el Modelo Relacional que se muestra en la sección **Documentos** de esta lectura, la idea es que podamos completar la documentación del Modelo Orientado a Documentos de dicha base de datos, esto con el objetivo de conocer con mayor exactitud la posible organización de sus datos.

Referencias

1. Choudhary, H. *Relation in database MongoDB*. Marzo de 2019. [En línea]: disponible en <https://www.youtube.com/watch?v=HEXHy4JyDFc>. [Accedido en 2020].
2. Datademia. *¿Qué es MongoDB?* Diciembre de 2020. [En línea]: disponible en <https://www.youtube.com/watch?v=AFBKaeblA2s&t=13s>. [Accedido en 2023].
3. MitoCode. *Curso MongoDB - 6 Colecciones y Documentos*. Abril de 2021. [En línea]: disponible en <https://www.youtube.com/watch?v=WVnxpWNU-qs&t=48s>. [Accedido en 2023].
4. MitoCode. *Curso MongoDB - 10 Modelamiento NoSQL*. Mayo de 2021. [En línea]: disponible en <https://www.youtube.com/watch?v=GNJS4DaKNp4>. [Accedido en 2023].
5. MongoDB. *Databases and Collections*. MongoDB Manual. [En línea]: disponible en <https://www.mongodb.com/docs/manual/core/databases-and-collections/>. [Accedido en 2023].
6. MongoDB. *Documents*. MongoDB Manual. [En línea]: disponible en <https://www.mongodb.com/docs/manual/core/document/>. [Accedido en 2023].
7. Raboy, N. *Model Your Relational Database Data as NoSQL Document*. Julio de 2016. [En línea]: disponible en <https://www.youtube.com/watch?v=Hdi1awDJi3Q>. [Accedido en 2020].
8. Reiner, B. *Transitioning from Relational Databases to MongoDB - Data Models*. Abril de 2017. [En línea]: disponible en <https://www.mongodb.com/blog/post/transitioning-from-relational-databases-to-mongodb>. [Accedido en 2020].
9. Trivedi, A. *Mapping Relational Databases and SQL to MongoDB*. Febrero de 2014. [En línea]: disponible en <https://code.tutsplus.com/articles/mapping-relational-databases-and-sql-to-mongodb--net-35650>. [Accedido en 2020].